



Proyecto Final: Motores paralelos de Mancala (Kalah) y despliegue en Kubernetes Infraestructuras paralelas y distribuidas

Carlos Andres Delgado S, Msc
`carlos.andres.delgado@correounivalle.edu.co`

Junio 2026

1. Descripción del Problema

Este proyecto integra los dos hilos conductores del curso: **paralelización con instrumentación** sobre un problema de cómputo intensivo y **despliegue distribuido** de la solución sobre Kubernetes en la nube. El problema escogido es la construcción de un motor de juego para **Mancala**, en su variante **Kalah**, resuelto con búsqueda adversaria exhaustiva: **Minimax** con **poda Alfa-Beta**. El motor se implementa, se paraleliza y se mide sobre un conjunto fijo de posiciones de prueba.

Mancala es una familia milenaria de juegos de tablero de la cuenca del Mediterráneo y África. Kalah es la variante moderna más estudiada en inteligencia artificial: dos jugadores, dos filas de seis hoyos, dos almacenes (*kalahas*) y reglas deterministas de información perfecta, lo cual la hace un banco de pruebas ideal para algoritmos de búsqueda en árboles de juego (ver <https://en.wikipedia.org/wiki/Mancala>).

El espacio de estados de Kalah crece exponencialmente con la profundidad de búsqueda. Una IA que evalúe muchas jugadas por segundo se beneficia directamente de la paralelización del recorrido del árbol. La poda Alfa-Beta agrega un reto adicional: distribuir el trabajo entre hilos rompe el orden secuencial de las cotas y suele *perder podas* respecto a la versión secuencial. Cuantificar esa pérdida y diseñar la estrategia de paralelización para minimizarla es uno de los ejes pedagógicos del proyecto. Se le pide al estudiante:

1. Implementar en C++ un **motor de búsqueda** basado en Minimax con poda Alfa-Beta. El motor incluye una representación del tablero, un generador de movimientos legales y una función de evaluación heurística.
2. Paralelizar el motor con OpenMP, **instrumentando** tiempo, *speedup* y eficiencia con una metodología común de medición.

3. Exponer el sistema como servicio HTTP mediante un **wrapper en Python** con FastAPI o Flask.
4. Empaquetarlo en contenedores y orquestarlo con Kubernetes, tanto en local como en un servicio gestionado en la nube.
5. Comparar de forma cuantitativa la ejecución **local (variando hilos en una sola instancia)** contra la ejecución **distribuida (variando réplicas del backend en Kubernetes)** sobre las mismas posiciones de prueba.

1.1. Reglas de Kalah que se deben implementar

Para que las mediciones entre grupos sean comparables, se exige la variante estándar Kalah(6,4):

- Tablero con dos filas de seis hoyos, cuatro semillas inicialmente en cada hoyo.
- Cada jugador tiene un *kalaha* (almacén) a su derecha.
- En su turno, el jugador toma todas las semillas de uno de sus hoyos y las distribuye una a una en sentido antihorario, incluyendo su propio kalaha pero saltando el del oponente.
- Si la última semilla cae en el kalaha propio, el jugador repite turno.
- Si la última semilla cae en un hoyo propio vacío y el hoyo opuesto del rival contiene semillas, el jugador captura ambas hacia su kalaha.
- El juego termina cuando uno de los lados queda vacío; el jugador con más semillas en su kalaha gana.

1.2. Primer algoritmo: Minimax con poda Alfa-Beta

El primer motor evalúa cada jugada legal mediante una búsqueda en profundidad fija (parámetro `depth`) sobre el árbol de juego. La función de evaluación heurística mínima exigida combina:

$$h(\text{estado}) = (\text{kalaha_propio} - \text{kalaha_rival}) + \alpha \cdot (\text{semillas_lado_propio} - \text{semillas_lado_rival})$$

con $\alpha \in [0, 1]$ definido por el grupo. La poda Alfa-Beta debe implementarse correctamente: árboles podados deben producir el mismo movimiento óptimo que el Minimax sin poda a igual profundidad. Esta equivalencia es uno de los criterios de corrección.

2. Arquitectura del Sistema

2.1. Separación estricta en contenedores

Cada componente del sistema debe vivir en su propio contenedor. No se permite empaquetar el motor dentro del mismo contenedor del backend, ni servir el frontend desde el

proceso del backend, ni embeber la base de datos en cualquier otro contenedor. La razón es pedagógica y operativa: el proyecto evalúa la capacidad de **orquestrar servicios** con Kubernetes, lo cual exige que cada servicio sea desplegable, escalable y reinicializable de forma independiente.

Los componentes son:

- **Motor (C++ / OpenMP) – contenedor propio:** proceso de larga vida que expone un *endpoint* interno (HTTP, gRPC o equivalente) sobre la red del clúster. Recibe un estado del tablero y una profundidad de búsqueda y devuelve la jugada óptima. Es el componente paralelizado y el sujeto principal de la instrumentación. **No** se enlaza en el mismo proceso del backend mediante `pybind11` o `ctypes`: la separación debe ser a nivel de contenedor, no a nivel de proceso.
- **Backend (Python + FastAPI o Flask) – contenedor propio:** *wrapper* HTTP que expone la API pública al frontend, valida la entrada y delega el cálculo al contenedor del motor a través de la red interna del clúster.
- **Frontend (HTML/JS o React) – contenedor propio:** servidor web (por ejemplo `nginx`) que entrega los archivos estáticos del cliente. El cliente se ejecuta en el navegador del usuario y consume la API pública del backend.
- **Base de datos (opcional) – contenedor propio:** si el grupo desea persistir partidas, *rankings* o resultados de *benchmarks*, agrega un contenedor de PostgreSQL o MySQL con su propio `PersistentVolume Claim`. No es obligatorio.

2.2. Diagrama de orquestación

La figura siguiente muestra la interacción entre los servicios. Las flechas indican el sentido de las peticiones; cada caja es un **Service** de Kubernetes con uno o más *pods* detrás.

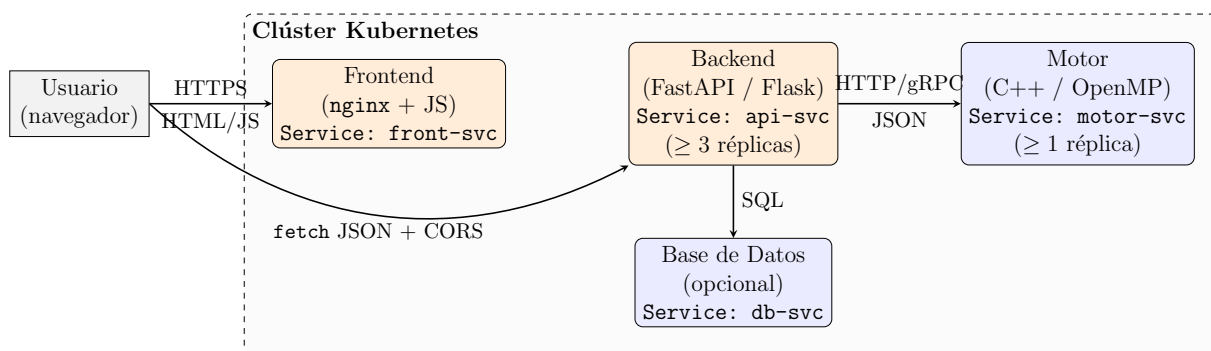


Figura 1: Orquestación de servicios. El navegador descarga el frontend desde `front-svc` y luego llama directamente a `api-svc` mediante peticiones JSON con CORS habilitado. El backend delega el cálculo de la jugada al motor a través de la red interna del clúster.

2.3. API REST: contrato y serialización

La comunicación frontend → backend → motor se realiza por HTTP con **cargas útiles serializadas en JSON**. No se aceptan formatos binarios opacos, parámetros de consulta para estados complejos, ni mezclas de `form-data` con JSON. Reglas mínimas:

- Toda petición y respuesta usan `Content-Type: application/json; charset=utf-8`.
- Los *schemas* de entrada y salida están explícitamente definidos. En FastAPI esto se hace con modelos `pydantic`; en Flask con `marshmallow`, `pydantic` o equivalente. La validación rechaza con HTTP 422 cualquier petición malformada.
- Los códigos de estado HTTP se respetan: 200 éxito, 400 entrada inválida del cliente, 422 fallo de validación del *schema*, 500 error interno, 503 motor no disponible.
- El tablero se serializa como un arreglo de 14 enteros (12 hoyos + 2 *kalahas*) en orden canónico y documentado, para que cualquier cliente externo pueda interpretarlo sin ambigüedad.

Endpoints mínimos del backend:

- POST `/move`: recibe `{ board, side, depth, threads }` y responde con `{ move, evaluation, elapsed_ms, stats }`. El bloque `stats` contiene las métricas del motor (nodos visitados, podas efectuadas).
- GET `/healthz`: *liveness probe* para Kubernetes; responde 200 si el proceso está vivo.
- GET `/readyz`: *readiness probe*; responde 200 solo si el motor está accesible.
- GET `/metrics`: métricas agregadas del motor (texto plano o Prometheus).

Ejemplo de respuesta de `/move`:

```
{
  "move": 3,
  "evaluation": 7,
  "elapsed_ms": 124,
  "stats": {
    "nodes": 1845210,
    "prunes": 312088
  },
  "threads_used": 4
}
```

2.4. CORS y seguridad de origen cruzado

Como el frontend y el backend se sirven en **orígenes distintos** (dominios o puertos diferentes), el navegador aplica la política de *Same-Origin Policy* y bloquea por defecto las peticiones `fetch` hacia el backend. El backend debe declarar explícitamente una política de **CORS** (*Cross-Origin Resource Sharing*). Reglas exigidas:

- Configurar el *middleware* de CORS en el backend (`fastapi.middleware.cors.CORSMiddleware` o `flask-cors`).
- Especificar de forma explícita los orígenes permitidos (dominio del frontend en local y en la nube). **No** se acepta usar el comodín "*" en producción.

- Habilitar únicamente los métodos HTTP que la API realmente usa (GET, POST, OPTIONS).
- Manejar correctamente las peticiones *preflight* OPTIONS que el navegador envía antes de cualquier POST con Content-Type: application/json.
- Documentar en el informe la configuración de CORS y por qué se eligieron esos orígenes.

Ejemplo en FastAPI:

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:8080",
        "https://mancala.midominio.cloud",
    ],
    allow_methods=["GET", "POST", "OPTIONS"],
    allow_headers=["Content-Type"],
)
```

3. Paralelización del Motor con OpenMP

El núcleo del proyecto es la paralelización del motor con OpenMP. El árbol de Alfa-Beta tiene dependencias de poda que penalizan el reparto del trabajo entre hilos, así que la estrategia elegida debe discutirse y medirse. La versión paralela debe producir el mismo movimiento óptimo que la versión secuencial sobre las mismas posiciones de prueba.

3.1. Estrategias para Alfa-Beta

Se exige al menos **una** de las siguientes estrategias implementada y discutida:

1. **Paralelismo a la raíz** (*root parallelism*): se reparte la lista de movimientos legales del nodo raíz entre hilos con `#pragma omp parallel for`, cada uno ejecuta una búsqueda Alfa-Beta secuencial sobre su subárbol. Es la estrategia más simple y la base mínima.
2. **Young Brothers Wait Concept (YBWC)**: explora el primer hijo secuencialmente para obtener una buena cota β y luego paraleliza los hermanos restantes. Reduce la pérdida de podas del esquema anterior.
3. **Principal Variation Splitting (PVS)**: variante donde la búsqueda de la variante principal se hace secuencial y los nodos de *re-search* se distribuyen en paralelo.

Cualquiera que sea la estrategia, deben discutir explícitamente el **costo de la sincronización** de las cotas α y β entre hilos y la pérdida de podas respecto al algoritmo secuencial.

3.2. Instrumentación obligatoria

El motor debe poder ejecutarse en modo *benchmark*, sin necesidad de pasar por el backend HTTP, leyendo un conjunto fijo de posiciones de prueba.

Métricas comunes

- Tiempo de pared (`omp_get_wtime` o `std::chrono::steady_clock`).
- *Speedup* $S(p) = T(1)/T(p)$.
- Eficiencia $E(p) = S(p)/p$.

Métricas específicas de Alfa-Beta

Por cada par (profundidad, hilos):

- Número de nodos explorados.
- Número de podas Alfa-Beta efectuadas.

Barrido experimental

Las mediciones deben hacerse para $p \in \{1, 2, 4, 8\}$ hilos como mínimo y en al menos dos profundidades distintas (por ejemplo `depth=8` y `depth=12`). Los resultados se reportan en tablas y en al menos una gráfica de *speedup* vs. número de hilos.

3.3. Herramientas de profiling

Se exige el uso, documentado con capturas o registros, de al menos dos de las siguientes herramientas:

- `perf stat` y `perf record` para contadores de hardware (instrucciones, ciclos, *branch misses*).
- `htop` o `top` para ocupación efectiva de núcleos durante la búsqueda paralela.
- `time`, `/usr/bin/time -v` para tiempo y *maximum resident set size*.
- `valgrind --tool=callgrind` (opcional, útil para detectar puntos calientes).

Un ejemplo de invocación esperada del benchmark es:

```
# Secuencial
OMP_NUM_THREADS=1 ./mancala_bench --depth 12 \
  --positions tests/suite.txt

# Paralelo con perf stat
OMP_NUM_THREADS=8 perf stat -e cycles,instructions,cache-misses \
  ./mancala_bench --depth 12 \
  --positions tests/suite.txt
```

4. Solución Local

El despliegue local debe servir como base reproducible antes de migrar a la nube. Se exigen **ambos** entornos:

4.1. Entorno de desarrollo con Docker Compose

`docker-compose.yml` debe levantar la aplicación completa con un solo comando, suficiente para que un evaluador externo clone el repositorio y ejecute `docker compose up` sin pasos manuales adicionales.

```
services:
  motor-backend:
    build: ./backend
    ports:
      - "8000:8000"
    environment:
      OMP_NUM_THREADS: "4"

  frontend:
    build: ./frontend
    ports:
      - "8080:80"
    depends_on:
      - motor-backend
```

4.2. Entorno de orquestación con Kubernetes local

Se debe entregar un conjunto de manifiestos YAML que levanten la aplicación en un clúster local (`kind`, `minikube` o `k3d`). Como mínimo:

- `Deployment` para el backend con ≥ 3 réplicas.
- `Service` (tipo `ClusterIP` interno y `NodePort` o `LoadBalancer` para exposición).
- `Deployment` y `Service` para el frontend.
- `ConfigMap` para variables del motor (`OMP_NUM_THREADS`, profundidad por defecto).
- *Probes* `liveness` y `readiness` apuntando a `/healthz`.

5. Solución en la Nube

La solución se despliega sobre un **servicio gestionado de Kubernetes**: AWS EKS, Azure AKS, GCP GKE o equivalente. La elección del proveedor es libre, pero el despliegue debe cumplir dos condiciones **innegociables**:

1. Toda la configuración del clúster y de la aplicación debe quedar declarada en archivos **YAML** (manifiestos de Kubernetes y, opcionalmente, Helm o Kustomize). **No se acepta** configurar el despliegue exclusivamente desde la consola web del proveedor.
2. El balanceo de carga entre las réplicas del backend lo maneja el propio **Service** de Kubernetes (tipo **LoadBalancer** o vía **Ingress**). No se exige un balanceador externo adicional.

5.1. Componentes en la nube

- **Backend replicado:** ≥ 3 réplicas del wrapper Python + motor C++ servidas por el **Service** de Kubernetes.
- **Frontend:** una sola réplica, expuesta vía **Ingress** o **Service** de tipo **LoadBalancer**.
- **Imágenes en un registro:** las imágenes del backend y del frontend deben estar en un registro accesible por el clúster (Docker Hub, GHCR, ECR, ACR, GCR). El **tag** debe ser inmutable (no usar **latest** en producción).
- **Recursos declarados:** cada contenedor debe declarar **requests** y **limits** de CPU y memoria. Esto es indispensable para que el análisis comparativo sea honesto.

6. Integración Continua y Calidad de Código

La integración continua y el análisis de calidad se hacen mediante **GitHub Actions**, descritos íntegramente en archivos YAML dentro de `.github/workflows/`. Como mínimo:

- Compilación del motor C++ y ejecución de pruebas unitarias.
- Construcción y publicación de las imágenes Docker al registro elegido.
- Análisis estático de calidad con **SonarQube** (o SonarCloud) invocado desde el *workflow* YAML mediante el **sonar-scanner**. **No se acepta** la integración mediante el *plugin* de GitHub que se instala desde el *marketplace*; toda la integración debe estar versionada en el repositorio.

Un esqueleto válido del paso de Sonar dentro de un *workflow* sería:

```
- name: SonarQube Scan
  uses: sonarsource/sonarqube-scan-action@v2
  env:
    SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
    SONAR_HOST_URL: ${ secrets.SONAR_HOST_URL }
```

7. Análisis Comparativo: Local vs. Nube

La comparación cuantitativa entre ambos entornos se mantiene **acotada** para que sea reproducible dentro del tiempo del proyecto. Se conserva **la misma imagen del motor** y se varía únicamente la capa de orquestación.

7.1. Configuración del experimento

- **Local:** una sola instancia del backend ejecutándose en una máquina física o VM, variando `OMP_NUM_THREADS` $\in \{1, 2, 4, 8\}$ a profundidad fija.
- **Nube:** el mismo backend desplegado en Kubernetes con dos configuraciones de réplicas, $r \in \{1, 3\}$. Se fija `OMP_NUM_THREADS` a un valor moderado (por ejemplo 2) y se varía r .
- **Carga sintética:** se generan peticiones concurrentes con una herramienta de *load testing* como `wrk`, `k6` o `ab`. Cada petición envía un estado de tablero y una profundidad fija.

7.2. Métricas a reportar

- **Latencia** por petición: percentiles p50 y p95.
- **Throughput:** peticiones por segundo sostenidas.

El informe incluye una tabla con los dos entornos y los dos criterios, cerrada con una **observación cualitativa de una o dos frases** sobre cuándo conviene escalar verticalmente (más hilos por pod) y cuándo horizontalmente (más réplicas), apoyada en los números obtenidos. No se pide análisis profundo del comportamiento del **Service** bajo carga creciente.

8. Estructura del Informe

8.1. Formato y ubicación

Todo informe debe estar en formato **Markdown** dentro de la carpeta `docs/` del repositorio. Esta es la **única** forma aceptada de entrega del informe. En particular:

- **No se aceptan** archivos PDF, DOCX, ODT, RTF, Notion, Google Docs, ni cualquier otro formato distinto a Markdown. Si aparecen en el repositorio, serán **ignorados** por la evaluación.
- **No se aceptan** enlaces a Google Drive, OneDrive, Dropbox, Notion, ni a documentos hospedados fuera del repositorio. Todo lo evaluable debe estar versionado dentro del repositorio entregado.
- **No se aceptan imágenes** (`.png`, `.jpg`, `.svg`) como sustituto de un diagrama. Capturas de pantalla de la consola, dashboards o resultados experimentales sí se aceptan, pero los **diagramas de arquitectura, flujos, secuencia o despliegue deben estar escritos en Mermaid** embebidos en el Markdown.

Mermaid se renderiza nativamente en GitHub dentro de bloques de código con la etiqueta `mermaid`, lo cual mantiene los diagramas versionables, diffeables y editables sin herramientas externas. Ejemplo:

```

```mermaid
flowchart LR
 user[Usuario] --> front[Frontend nginx]
 user -->|fetch JSON + CORS| api[Backend FastAPI]
 api -->|HTTP/gRPC| motor[Motor C++/OpenMP]
 api --> db[(Base de Datos)]
```

```

8.2. Estructura del informe en docs/

El informe **no es un único archivo**: se divide en **ocho archivos Markdown temáticos** más un índice navegable en `docs/README.md`. Esta partición facilita el trabajo en paralelo entre los integrantes del grupo, la evaluación granular por rúbrica y los *diffs* limpios en el repositorio.

La estructura obligatoria de `docs/` es:

```

docs/
|-- README.md                # índice navegable + tabla criterio -> archivo
|-- 01-arquitectura.md      # componentes, diagrama Mermaid, API REST, CORS
|-- 02-motor.md             # reglas Kalah(6,4), Minimax + Alfa-Beta, pseudocodi
|-- 03-paralelizacion.md    # estrategia OpenMP + instrumentacion local
|-- 04-despliegue-local.md  # docker-compose + Kubernetes local (kind/minikube)
|-- 05-despliegue-nube.md   # manifiestos YAML, proveedor, evidencias kubectl
|-- 06-cicd.md              # workflows GitHub Actions + SonarQube
|-- 07-analisis-comparativo.md # local vs nube: p50/p95/p99, throughput, recursos
`-- 08-conclusiones.md       # limitaciones, retos, trabajo futuro

```

Reglas de la división

1. `docs/README.md` es **obligatorio** y contiene: (a) un índice con enlaces relativos a los ocho archivos, y (b) una tabla que mapea cada criterio de la rúbrica al archivo donde se atiende. Sin este índice, la evaluación recorre los archivos en orden numérico y omite cualquier contenido que no se ubique sin ambigüedad.
2. La numeración con prefijo (01-, 02-, ...) es obligatoria y define el orden de lectura natural en GitHub.
3. Cada archivo es **autocontenido**: puede leerse de forma aislada y, cuando necesite referenciar a otro, enlaza con ruta relativa (por ejemplo, [ver paralelización](03-paralelizacion.md)).
4. Los **diagramas Mermaid** viven en el archivo donde aplican; no se centralizan en un archivo aparte.
5. **Está prohibido** agregar archivos Markdown adicionales en `docs/` fuera de los ocho listados: cualquier archivo extra es ignorado por la evaluación.

Contenido mínimo de cada archivo

1. `01-arquitectura.md`: visión general del sistema, diagrama Mermaid de orquestación de servicios, descripción de los contenedores (motor, backend, frontend, BD opcional) y su interacción, contrato de la API REST con *endpoints* y *schemas* JSON, política de CORS.
2. `02-motor.md`: reglas de Kalah(6,4) implementadas, función de evaluación heurística, pseudocódigo de Minimax con poda Alfa-Beta, suite de pruebas unitarias y cómo ejecutarla, evidencia de que Alfa-Beta produce el mismo movimiento óptimo que Minimax a igual profundidad sobre el conjunto de posiciones de prueba.
3. `03-paralelizacion.md`: estrategia OpenMP elegida (*root*, YBWC o PVS) con justificación, discusión del costo de sincronización de las cotas α y β , tablas de $T(p)$, $S(p)$, $E(p)$ para $p \in \{1, 2, 4, 8\}$ en al menos dos profundidades, gráficas de *speedup*, capturas de `perf` / `htop` / `time`.
4. `04-despliegue-local.md`: `Dockerfile` de cada componente, `docker-compose.yml` comentado, manifiestos YAML para Kubernetes local (*kind* / *minikube* / *k3d*), diagrama Mermaid del flujo de contenedores, comandos para reproducir el despliegue desde cero.
5. `05-despliegue-nube.md`: proveedor de Kubernetes gestionado escogido (EKS / AKS / GKE), manifiestos YAML versionados, evidencia de `kubectl get pods,svc, deploy`, diagrama Mermaid del despliegue, declaración explícita de `requests` y `limits` y justificación.
6. `06-cicd.md`: *workflows* de `.github/workflows/`, integración con SonarQube declarada en YAML (no *plugin*), diagrama Mermaid del *pipeline*, evidencia de ejecuciones exitosas y *quality gate* de Sonar.
7. `07-analisis-comparativo.md`: experimento descrito en la sección Análisis Comparativo, tabla embebida en el Markdown con la latencia (p50/p95) y el *throughput* en local (variando hilos) y en la nube (réplicas $r \in \{1, 3\}$), cierre cualitativo sobre escalado vertical vs. horizontal.
8. `08-conclusiones.md`: limitaciones encontradas, retos durante el desarrollo y cómo se resolvieron, recomendaciones de mejoras futuras, lecciones aprendidas.

Mapeo a la rúbrica

| Criterio de la rúbrica | Archivo donde se evalúa |
|--------------------------------------|----------------------------|
| Motor de Mancala: corrección | 02-motor.md |
| Paralelización con OpenMP | 03-paralelizacion.md |
| Instrumentación local | 03-paralelizacion.md |
| Separación de componentes | 01-arquitectura.md |
| Despliegue local | 04-despliegue-local.md |
| Despliegue en la nube con Kubernetes | 05-despliegue-nube.md |
| CI/CD y calidad de código | 06-cicd.md |
| Análisis comparativo local vs. nube | 07-analisis-comparativo.md |
| Claridad de explicaciones | transversal a todos |
| Conclusiones | 08-conclusiones.md |

9. Entrega

9.1. Fechas y mecanismo de entrega

1. La fecha de entrega es **miércoles, 10 de junio de 2026 a las 17:58:59** mediante *push* a la rama principal (**main** o **master**) del repositorio asignado por **GitHub Classroom**. **No se aceptan entregas por ningún otro medio** (correo, USB, mensajería, repositorios alternos) ni después de la fecha; se considera la fecha del último *commit* a la rama principal antes del cierre.
2. La **sustentación** es el **jueves, 11 de junio de 2026** en el horario de clase. La asistencia y participación activa de **todos** los integrantes es obligatoria. La inasistencia individual implica nota **0.0** para el integrante ausente, sin afectar al resto del grupo.
3. **No hay inscripción manual** de grupos: los grupos se conforman a través del *roster* de GitHub Classroom al momento de aceptar la asignación. Los grupos son de **hasta 5 estudiantes**. Grupos con menos integrantes están permitidos, pero la carga de trabajo y la rúbrica no se reducen. Cualquier conformación distinta debe resolverse antes de aceptar la asignación.

9.2. Formato del informe y de la documentación

1. **Todo informe debe estar en formato MARKDOWN** dentro de la carpeta `docs/` del repositorio. **No se acepta otro formato** (PDF, DOCX, ODT, RTF, Pages, Notion, Google Docs, HTML compilado) ni archivos por fuera de esa carpeta; cualquier archivo en otro formato será **ignorado** por la evaluación. Puede revisar la guía oficial en <https://docs.github.com/es/get-started/writing-on-github/getting-started-with-writing-and-formatting-on-github/basic-writing-and-formatting-syntax>.
2. **No se aceptan enlaces** a Google Drive, OneDrive, Dropbox, Notion, Confluence, sitios externos ni a cualquier documento hospedado por fuera del repositorio. Todo lo evaluable debe estar versionado dentro del mismo repositorio entregado.

3. La **notación matemática** (fórmulas de *speedup*, eficiencia, tablas analíticas, complejidad, etc.) debe escribirse en **formato L^AT_EX embebido en el MARKDOWN**. No se **acepta** en otro formato ni como imagen. Puede revisar la guía oficial en <https://docs.github.com/es/get-started/writing-on-github/working-with-advanced-formatting/writing-mathematical-expressions>.
4. **Los diagramas** (arquitectura, flujo de contenedores, despliegue en Kubernetes, *pipeline* de CI/CD, secuencia de llamadas, etc.) deben generarse con **Mermaid** embebido en el MARKDOWN. No se **aceptan imágenes insertadas** (.png, .jpg, .svg) como sustituto de un diagrama. Puede revisar la guía oficial en <https://docs.github.com/en/get-started/writing-on-github/working-with-advanced-formatting/creating-diagrams>.
5. **Las capturas de pantalla** (consola, *htop*, *perf*, *dashboard* del proveedor de nube, *kubectl get pods*, resultados de SonarQube, *benchmarks* con *wrk* o *k6*) sí se aceptan como evidencia experimental dentro del informe, referenciadas con rutas relativas desde el MARKDOWN. Las capturas **nunca** sustituyen un diagrama: son evidencia de ejecución, no de diseño.

9.3. Identificación del grupo y del repositorio

1. El archivo README.md en la raíz del repositorio debe contener: **nombres completos, códigos estudiantiles y correos institucionales** de todos los integrantes del grupo, además de instrucciones mínimas para construir y ejecutar la aplicación localmente y un enlace al informe en docs/. No completar el README.md se penaliza con un **10 %** sobre la nota final del proyecto.
2. Solo se considera el código de la rama principal (*main* o *master*); el contenido de ramas secundarias **no** se evalúa.
3. Los *commits* deben hacerse desde la cuenta personal de GitHub de cada integrante (no *commits* colectivos desde una sola cuenta). Esta es la base para el factor de aporte al repositorio.

9.4. Estructura obligatoria del repositorio

El repositorio debe organizarse, **como mínimo**, con la siguiente estructura. Los archivos fuera de estas carpetas no se evalúan.

```
.
|-- README.md                # nombres, codigos, correos, build
|-- docs/                    # informe en 8 archivos Markdown + Mermaid
|   |-- README.md            # indice + mapeo criterio -> archivo
|   |-- 01-arquitectura.md
|   |-- 02-motor.md
|   |-- 03-paralelizacion.md
|   |-- 04-despliegue-local.md
|   |-- 05-despliegue-nube.md
```

```

| |-- 06-cicd.md
| |-- 07-analisis-comparativo.md
| |-- 08-conclusiones.md
|-- motor/                                # contenedor 1: motor C++/OpenMP
| |-- src/
| |-- tests/
| |-- bench/
| |-- Dockerfile
|-- backend/                              # contenedor 2: wrapper FastAPI/Flask
| |-- app/
| |-- tests/
| |-- Dockerfile
|-- frontend/                             # contenedor 3: cliente web (nginx)
| |-- Dockerfile
|-- deploy/
| |-- local/                              # docker-compose + manifiestos kind/minikube
| |-- cloud/                              # manifiestos YAML del despliegue en la nube
|-- .github/
| |-- workflows/                          # pipelines CI/CD + SonarQube por YAML

```

9.5. Reglas de implementación que aplican a la nota

1. **Cada componente del sistema en un contenedor aparte:** motor, backend, frontend y — si se incluye — base de datos. **No se permite** acoplar el motor al backend en el mismo proceso (pybind11, ctypes, mismo Dockerfile); la separación debe ser a nivel de contenedor y la comunicación entre ellos por la red del clúster.
2. **Toda la configuración de Kubernetes debe estar versionada en YAML** dentro de `deploy/cloud/`. **No se acepta** configurar el clúster exclusivamente desde la consola web del proveedor: si algo solo funciona “porque se hizo *click*”, no cuenta como entregable. Pueden complementar con Helm o Kustomize, pero la fuente de verdad debe estar en el repositorio.
3. **La integración con SonarQube debe estar declarada en YAML** dentro de un *workflow* de `.github/workflows/`. **No se acepta** la integración instalada como *plugin* desde el *marketplace* de GitHub.
4. **Las cargas útiles del API deben estar serializadas en JSON** con *schemas* explícitos (pydantic, marshmallow o equivalente), validación que rechace con HTTP 422 y códigos de estado HTTP estándar. **No se acepta** form-data para los *endpoints* del motor ni respuestas binarias opacas.
5. **El backend debe declarar explícitamente una política de CORS** con los orígenes permitidos del frontend (en local y en la nube). **No se acepta** usar el comodín "*" ni omitir el manejo de la *preflight* OPTIONS.
6. Archivos generados o temporales (`build/`, `node_modules/`, binarios, PDFs compilados, `_minted`, `__pycache__`) deben estar listados en `.gitignore` y **no** cuentan como aporte al

repositorio.

7. Si una decisión técnica se aparta de lo descrito en este documento, debe estar **justificada explícitamente en el informe**. Decisiones no documentadas se interpretan como incumplimiento del requisito.

10. Rúbricas y Evaluación

10.1. Rúbrica de evaluación

| Criterio | Nivel 0 (0 pts) | Nivel 1 (5 pts) | Nivel 2 (10 pts) | Nivel 3 (15 pts) |
|-------------------------------------|----------------------|-----------------------------------|-------------------------------|---|
| Motor de Mancala: corrección | Sin motor funcional | Solo se cumple una entre A, B o C | Se cumplen dos entre A, B o C | A) Reglas de Kallah(6,4) implementadas correctamente con turno extra y captura; B) Alfa-Beta produce el mismo movimiento óptimo que Minimax a igual profundidad sobre el conjunto de prueba; C) Suite de pruebas unitarias incluida y pasando |
| Paralelización con OpenMP | Sin versión paralela | Sólo se cumple una entre A, B o C | Se cumplen dos entre A, B o C | A) Estrategia de paralelización implementada correctamente (root, YBWC o PVS); B) Costo de sincronización de las cotas α y β documentado; C) Resultados paralelos coinciden con los secuenciales sobre las mismas posiciones |

| Criterio | Nivel 0 (0 pts) | Nivel 1 (5 pts) | Nivel 2 (10 pts) | Nivel 3 (15 pts) |
|---|----------------------------------|-----------------------------------|-------------------------------|---|
| Instrumentación local | No hay mediciones | Sólo se cumple una entre A, B o C | Se cumplen dos entre A, B o C | A) Tablas de $T(p), S(p), E(p)$ para $p \in \{1, 2, 4, 8\}$ en al menos dos profundidades; B) Uso documentado de al menos dos herramientas (<code>perf</code> , <code>htop</code> , <code>time</code> , <code>valgrind</code>); C) Gráfica de <i>speedup</i> y discusión de la pérdida de podas en la versión paralela respecto a la secuencial |
| Separación de componentes | Sin separación clara | Sólo se cumple una entre A, B o C | Se cumplen dos entre A, B o C | A) Carpetas separadas <code>motor/</code> , <code>backend/</code> , <code>frontend/</code> ; B) API REST documentada (<code>/move</code> , <code>/healthz</code> , <code>/metrics</code>); C) Documentación de la arquitectura modular incluida |
| Despliegue local | No se realiza el proceso | Sólo se cumple una entre A, B o C | Se cumplen dos entre A, B o C | A) <code>Dockerfile</code> correctos para <code>backend</code> y <code>frontend</code> ; B) <code>docker-compose.yml</code> levanta toda la aplicación con un comando; C) Manifiestos YAML para Kubernetes local funcionales |
| Despliegue en la nube con Kubernetes | No utiliza Kubernetes en la nube | Sólo se cumple una entre A, B o C | Se cumplen dos entre A, B o C | A) Toda la configuración versionada en YAML (no solo consola web); B) ≥ 3 réplicas del backend gestionadas por el <code>Service</code> de Kubernetes; C) <code>requests</code> y <code>limits</code> declarados en cada contenedor |

| Criterio | Nivel 0 (0 pts) | Nivel 1 (5 pts) | Nivel 2 (10 pts) | Nivel 3 (15 pts) |
|--|-----------------------------------|-----------------------------------|-------------------------------|---|
| CI/CD y calidad de código | No se incluye integración | Sólo se cumple una entre A, B o C | Se cumplen dos entre A, B o C | A) Pruebas automatizadas integradas; B) Construcción y publicación de imágenes desde GitHub Actions; C) Análisis con SonarQube/SonarCloud declarado en YAML, no por <i>plugin</i> de GitHub |
| Análisis comparativo local vs. nube | No se incluye análisis | Sólo se cumple una entre A, B o C | Se cumplen dos entre A, B o C | A) Latencia p50/p95 reportada en ambos entornos con la misma carga; B) <i>Throughput</i> sostenido reportado en ambos entornos; C) Observación cualitativa sobre cuándo escalar verticalmente vs. horizontalmente, anclada en los números obtenidos |
| Claridad de explicaciones: Informe | Explicaciones ausentes o confusas | Sólo se cumple una entre A, B o C | Se cumplen dos entre A, B o C | A) Explicaciones claras de la arquitectura y componentes; B) Diagramas e ilustraciones presentes; C) Terminología técnica correctamente definida y contextualizada |
| Conclusiones | Conclusiones ausentes | Sólo se cumple una entre A, B o C | Se cumplen dos entre A, B o C | A) Conclusiones basadas en métricas obtenidas; B) Reflexiones sobre retos y soluciones; C) Recomendaciones para mejoras futuras y lecciones aprendidas |

Las rúbricas del informe (claridad, análisis, conclusiones) dependen de la correcta implementación del motor, la paralelización y los despliegues. Si no se cumple alguna de esas tres bases, los criterios del informe se evalúan en **nivel 0**.

10.2. Sustentación

El proyecto debe ser sustentado por todos los miembros del grupo. La nota de la sustentación es un factor entre 0 y 1 que multiplica la nota grupal. Por ejemplo, si el grupo obtuvo 4.5 y el factor de sustentación fue 0.8, la nota final del grupo sería $4,5 \times 0,8 = 3,6$. La inasistencia a la sustentación implica nota 0.0 para el integrante ausente.

10.3. Aporte al repositorio

La nota individual se multiplica por un factor que refleja la contribución de cada estudiante al repositorio:

1. Se considera aporte al código: el código fuente del motor, el backend, el frontend, los manifiestos de despliegue (`Dockerfile`, YAML de Kubernetes), los *workflows* de CI/CD y la documentación.
2. Sólo se consideran los *commits* sobre la rama principal (`main` o `master`).
3. Los cambios deben ser significativos; cambios triviales de formato o comentarios son ignorados por la herramienta de análisis.
4. Archivos temporales o generados (carpetas `build/`, binarios, PDFs compilados) deben estar en `.gitignore` y no cuentan como aporte.

10.4. Cálculo de la nota

Las rúbricas suman un total de **150 puntos** ($10 \text{ criterios} \times 15 \text{ puntos}$). La nota grupal se calcula como:

$$\text{Nota grupal} = \left(\frac{\text{Puntos obtenidos}}{150} \right) \times 5$$

La nota individual se obtiene multiplicando la nota grupal por el factor de sustentación y por el factor de aporte al repositorio:

$$\text{Nota individual} = \text{Nota grupal} \times f_{\text{sust}} \times f_{\text{aporte}}$$